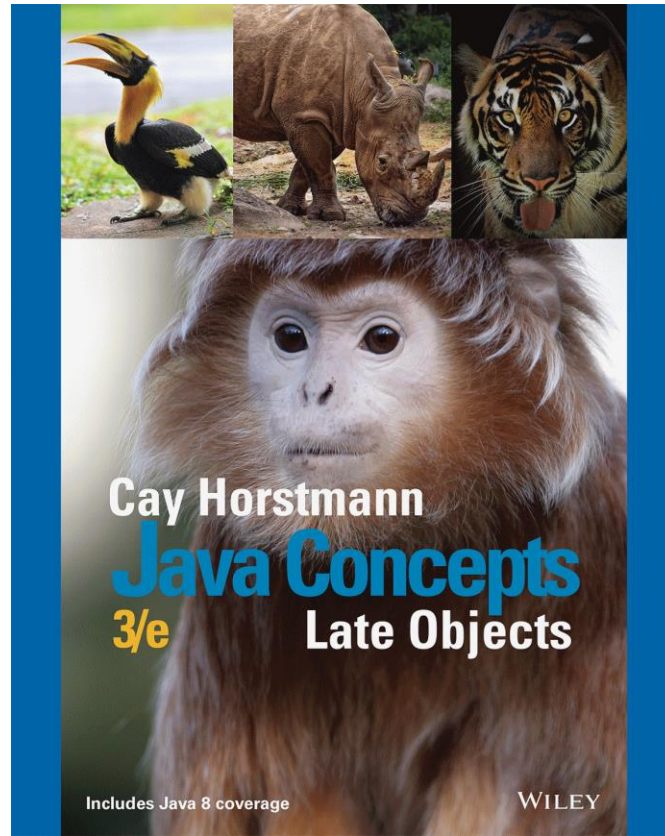


Chapter 7 - Input/Output and Exception Handling



Chapter Goals



- To read and write text files
- To process command line arguments
- To throw and catch exceptions
- To implement programs that propagate checked exceptions

Reading and Writing Text Files

- **Text** files are very commonly used to **store information**
 - Both **numbers** and **words** can be stored as text
 - They are the most '**portable**' types of data files
- The **Scanner** class can be used to read text files
 - We have used it to read from the keyboard
 - Reading from a file requires using the **File** class
- The **PrintWriter** class will be used to write text files
 - Using familiar **print**, **println** and **printf** tools

Text File Input

- Create an object of the `File` class
 - Pass it the name of the file to read in quotes

```
File inputFile = new File("input.txt");
```

- Then create an object of the `Scanner` class
 - Pass the constructor the new `File` object

```
Scanner in = new Scanner(inputFile);
```

- Then use `Scanner` methods such as:

- `next()`
- `nextLine()`
- `hasNextLine()`
- `hasNext()`
- `nextDouble()`
- `nextInt() ...`

```
while (in.hasNextLine())  
{  
    String line = in.nextLine();  
    // Process line;  
}
```

Text File Output

- Create an object of the `PrintWriter` class
 - Pass it the name of the file to write in quotes

```
PrintWriter out = new PrintWriter("output.txt");
```

- If `output.txt` exists, it will be emptied
- If `output.txt` does not exist, it will create an empty file
- `PrintWriter` is an enhanced version of `PrintStream`
- `System.out` is a `PrintStream` object!

```
System.out.println("Hello World!");
```

- Then use `PrintWriter` methods such as:

- `print()`
- `println()`
- `printf()`

```
out.println("Hello, World!");  
out.printf("Total: %8.2f\n", totalPrice);
```

Closing Files

- You must use the `close` method before file **reading** and **writing** is complete
- Closing a Scanner

```
while (in.hasNextLine())  
{  
    String line = in.nextLine();  
    // Process line;  
}  
in.close();
```

Your text may not be saved to the file until you use the `close` method!

- Closing a PrintWriter

```
out.println("Hello, World!");  
out.printf("Total: %8.2f\n", totalPrice);  
out.close();
```

Exceptions Preview

- One additional issue that we need to tackle:
 - If the input or output file for a `Scanner` doesn't exist, a `FileNotFoundException` occurs when the `Scanner` object is constructed.
 - The `PrintWriter` constructor can generate this exception if it cannot open the file for writing.
 - If the name is illegal or the user does not have the authority to create a file in the given location
 - Add two words to any method that uses File I/O

```
public static void main(String[] args) throws  
    FileNotFoundException
```

- Until you learn how to handle exceptions yourself

And an important **import** or two..

- Exception classes are part of the `java.io` package
 - Place the `import` directives at the beginning of the source file that will be using File I/O and exceptions

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintWriter;
import java.util.Scanner;

public class LineNumberer
{
    public void openFile() throws FileNotFoundException
    {
        . . .
    }
}
```


Total.java (1)

```
1  import java.io.File;
2  import java.io.FileNotFoundException;
3  import java.io.PrintWriter;
4  import java.util.Scanner;
5
6  /**
7   * This program reads a file with numbers, and writes the numbers to another
8   * file, lined up in a column and followed by their total.
9   */
10 public class Total
11 {
12     public static void main(String[] args) throws FileNotFoundException
13     {
14         // Prompt for the input and output file names
15
16         Scanner console = new Scanner(System.in);
17         System.out.print("Input file: ");
18         String inputFileName = console.next();
19         System.out.print("Output file: ");
20         String outputFileName = console.next();
21
22         // Construct the Scanner and PrintWriter objects for reading and writing
23
24         File inputFile = new File(inputFileName);
25         Scanner in = new Scanner(inputFile);
26         PrintWriter out = new PrintWriter(outputFileName);
```

More import statements required!
Some examples may use `import java.io.*;`

Note the throws clause

Total.java (2)

```
28 // Read the input and write the output
29
30 double total = 0;
31
32 while (in.hasNextDouble())
33 {
34     double value = in.nextDouble();
35     out.printf("%15.2f\n", value);
36     total = total + value;
37 }
38
39 out.printf("Total: %8.2f\n", total);
40
41 in.close();
42 out.close();
43 }
44 }
```

Don't forget to close the files
before your program ends.

Common Error

■ Backslashes in File Names

- When using a `String` literal for a file name with path information, you need to supply each backslash twice:

```
File inputFile = new File("c:\\homework\\input.dat");
```

- A single backslash inside a quoted string is the *escape character*, which means the next character is interpreted differently (for example, `'\n'` for a newline character)
- When a user supplies a filename into a program, the user should not type the backslash twice

Common Error

- Constructing a Scanner with a String

- When you construct a `PrintWriter` with a `String`, it writes to a file:

```
PrintWriter out = new PrintWriter("output.txt");
```

- This does *not* work for a `Scanner` object

```
Scanner in = new Scanner("input.txt"); // Error?
```

- It does *not* open a file. Instead, it simply reads through the `String` that you passed (`"input.txt"`)
 - To read from a file, pass `Scanner` a `File` object:

```
Scanner in = new Scanner(new File ("input.txt") );
```

- or

```
File myFile = new File("input.txt");  
Scanner in = new Scanner(myFile);
```

Special Topic: Reading Web Pages

- You can use a `Scanner` to read a web page

```
String address = "http://horstmann.com/index.html";  
URL pageLocation = new URL(address);  
Scanner in = new Scanner(pageLocation.openStream());
```

- Read the contents of the page with the `Scanner` in the usual way
- The `URL` constructor and the `openStream` method can throw an `IOException`, so tag the `main` method with `throws IOException`
- The `URL` class is contained in the `java.net` package

Special Topic: File Dialog Boxes

- You may want to use a file dialog box whenever users of your program need to pick a file.
- The `JFileChooser` class implements a file dialog box for the Swing user-interface toolkit.

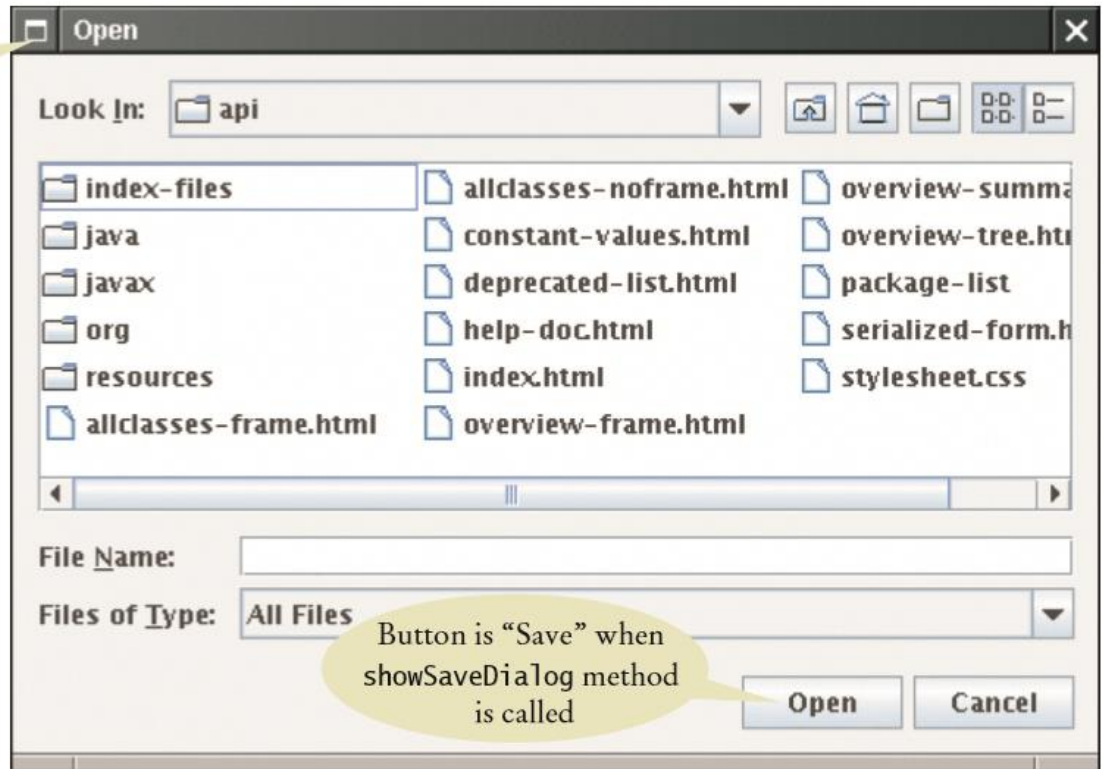
```
JFileChooser chooser = new JFileChooser();
Scanner in = null;
if (chooser.showOpenDialog(null) == JFileChooser.APPROVE_OPTION)
{
    File selectedFile = chooser.getSelectedFile();
    in = new Scanner(selectedFile);
    . . .
}
```

The `showOpenDialog` and `showSaveDialog` methods return either

- ✓ `JFileChooser.APPROVE_OPTION`, if the user has chosen a file, or
- ✓ `JFileChooser.CANCEL_OPTION`, if the user canceled the selection.

Special Topic: File Dialog Boxes

Call with
showOpenDialog
method



A JFileChooser Dialog Box

Special Topic: Reading and Writing Binary Data

- `Scanner` and `PrintWriter` classes read and write text files which are comprised of sequences of characters.
- Other files, such as **images**, are not made up of characters but of bytes.
- A byte is a fundamental storage unit in a computer—a number consisting of eight binary digits.
- A byte can represent unsigned integers between 0 and 255 or signed integers between -128 and 127
- The Java library has a different set of classes, called **streams**, for working with binary files

Special Topic: Reading and Writing Binary Data

- We cover a simple example of copying binary data from a web site to a file
- Use an `InputStream` to read binary data:

```
URL imageUrl =
    new URL("http://horstmann.com/java4everyone/duke.gif");
InputStream in = imageUrl.openStream();
FileOutputStream out = new FileOutputStream("duke.gif");
boolean done = false;
while (!done)
{
    int input = in.read(); // -1 or a byte between 0 and 255
    if (input == -1) { done = true; }
    else { out.write(input); }
}
```

Text Input and Output

- In the following sections, you will learn how to process text with complex contents, and you will learn how to cope with challenges that often occur with real data.
- Reading Words Example:

Mary had a little lamb

input

```
while (in.hasNext())  
{  
    String input = in.next();  
    System.out.println(input);  
}
```

output

Mary
had
a
little
lamb

Processing Text Input

- There are times when you want to read input by:
 - Each Word
 - Each Line
 - One Number
 - One Character
- Java provides methods of the **Scanner** and **String** classes to handle each situation
 - It does take some practice to mix them though!

Processing input is required for almost all types of programs that interact with the user.

Reading Words

- In the examples so far, we have read text one line at a time
- To read each word one at a time in a loop, use:
 - The `Scanner` object's `hasNext()` method to test if there is another word
 - The `Scanner` object's `next()` method to read one word

```
while (in.hasNext())  
{  
    String input = in.next();  
    System.out.println(input);  
}
```

Input:

Mary had a little lamb

Output:

Mary
had
a
little
lamb

White Space

- The Scanner's `next()` method has to decide where a word starts and ends.
- It uses simple rules:
 - It consumes all white space before the first character
 - It then reads characters until the first white space character is found or the end of the input is reached

White Space

- What is whitespace?
 - Characters used to separate:
 - Words
 - Lines

Common White Space

' '	Space
\n	NewLine
\r	Carriage Return
\t	Tab
\f	Form Feed

“Mary had a little lamb,\nher fleece was white as\snow”

The `useDelimiter` Method

- The `Scanner` class has a method to change the default set of delimiters used to separate words.

- The `useDelimiter` method takes a `String` that lists all of the characters you want to use as delimiters:

```
Scanner in = new Scanner(. . .);  
in.useDelimiter("[^A-Za-z]+");
```

- You can also pass a `String` in **regular expression** format inside the `String` parameter as in the example above.

- `[^A-Za-z]+` says that all characters that `^` not either `A-Z` uppercase letters A through Z or `a-z` lowercase a through z are delimiters.

- Search the Internet to learn more about regular expressions

Reading Characters

- There are no `hasNextChar()` or `nextChar()` methods of the `Scanner` class
- Instead, you can set the `Scanner` to use an 'empty' delimiter ("")

```
Scanner in = new Scanner(. . .);  
in.useDelimiter("");  
  
while (in.hasNext())  
{  
    char ch = in.next().charAt(0);  
    // Process each character  
}
```

- `next` returns a one-character `String`
- Use `charAt(0)` to extract the character from the `String` at index 0 to a `char` variable

Classifying Characters

- The `Character` class provides several useful methods to classify a character:
 - Pass them a `char` and they return a boolean

```
if ( Character.isDigit(ch) ) ...
```

Table 1 Character Testing Methods

Method	Examples of Accepted Characters
<code>isDigit</code>	0, 1, 2
<code>isLetter</code>	A, B, C, a, b, c
<code>isUpperCase</code>	A, B, C
<code>isLowerCase</code>	a, b, c
<code>isWhiteSpace</code>	space, newline, tab

Reading Lines

- Some text files are used as simple databases
 - Each line has a set of related pieces of information
 - This example is complicated by:
 - Some countries use two words
 - “United States”
- It would be better to read the entire line and process it using powerful `String` class methods

China 1330044605
India 1147995898
United States 303824646

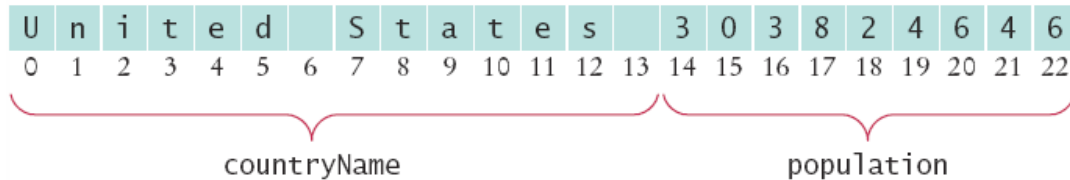
```
while (in.hasNextLine())  
{  
    String line = in.nextLine();  
    // Process each line  
}
```

- `nextLine()` reads one line and consumes the ending ‘\n’

U	n	i	t	e	d		S	t	a	t	e	s		3	0	3	8	2	4	6	4	6
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22

Breaking Up Each Line (String or Scanner method)

- Now we need to break up the line into two parts
 - Everything before the first digit is part of the country



- Get the index of the first digit with `Character.isdigit`

```
int i = 0;
while (!Character.isDigit(line.charAt(i))) { i++; }
```

- Use `String` methods to extract the two parts

```
String countryName = line.substring(0, i);
String population = line.substring(i);
// remove the trailing space in countryName
countryName = countryName.trim();
```

United States

303824646

trim removes white space at the beginning and the end.

Or Use Scanner Methods

▪ Instead of `String` methods, you can sometimes use `Scanner` methods to do the same tasks

- Read the line into a `String` variable United States 303824646
- Pass the `String` variable to a new `Scanner` object
- Use `Scanner hasNextInt` to find the numbers
- If not numbers, use `next` and concatenate words

```
Scanner lineScanner = new Scanner(line);

String countryName = lineScanner.next();
while (!lineScanner.hasNextInt())
{
    countryName = countryName + " " + lineScanner.next();
}
```

Remember the `next` method consumes white space.

Converting Strings to Numbers

- Strings can contain *digits*, not *numbers*
 - They must be converted to numeric types
 - 'Wrapper' classes provide a `parseInt` method

'3'	'0'	'3'	'8'	'2'	'4'	'6'	'4'	'6'
-----	-----	-----	-----	-----	-----	-----	-----	-----

```
String pop = "303824646";  
int populationValue = Integer.parseInt(pop);
```

'3'	'.'	'9'	'5'
-----	-----	-----	-----

```
String priceString = "3.95";  
int price = Double.parseDouble(priceString);
```

Caution: The argument must be a string containing only digits without any additional characters. Not even spaces are allowed! So... Use the `trim` method before parsing!

```
int populationValue = Integer.parseInt(pop.trim());
```

Safely Reading Numbers

- Scanner `nextInt` and `nextDouble` can get confused

- If the number is not properly formatted, an “Input Mismatch Exception” occurs

2 1 s t c e n t u r y

- Use the `hasNextInt` and `hasNextDouble` methods to test your input first

```
if (in.hasNextInt())  
{  
    int value = in.nextInt(); // safe  
}
```

- They will return `true` if digits are present
 - If true, `nextInt` and `nextDouble` will return a value
 - If not true, they would ‘throw’ an ‘Input Mismatch Exception’

Reading Other Number Types

- The `Scanner` class has methods to test and read almost all of the primitive types

Data Type	Test Method	Read Method
byte	<code>hasNextByte</code>	<code>nextByte</code>
short	<code>hasNextShort</code>	<code>nextShort</code>
int	<code>hasNextInt</code>	<code>nextInt</code>
long	<code>hasNextLong</code>	<code>nextLong</code>
float	<code>hasNextFloat</code>	<code>nextFloat</code>
double	<code>hasNextDouble</code>	<code>nextDouble</code>
boolean	<code>hasNextBoolean</code>	<code>nextBoolean</code>

- What is missing?
 - Right, no `char` methods!

Mixing Number, Word and Line Input

- `nextDouble` (and `nextInt...`) do not consume white space following a number
- This can be an issue when calling `nextLine` after reading a number
- There is a 'newline' at the end of each line
- After reading 1330044605 with `nextInt`
 - `nextLine` will read until the `\n` (an empty String)

China
1330044605
India

```
while (in.hasNextInt())  
{  
    String countryName = in.nextLine();  
    int population = in.nextInt();  
    in.nextLine();    // Consume the newline  
}
```



C h i n a \n 1 3 3 0 0 4 4 6 0 5 \n I n d i a \n

printf Format Specifier

- A format specifier has the following structure:
 - The first character is a `%`
 - Next, there are optional “flags” that modify the format, such as `-` to indicate left alignment. See Table 2 for the most common format flags
 - Next is the field width, the total number of characters in the field (including the spaces used for padding), followed by an optional precision for floating-point numbers
- The format specifier ends with the format type, such as `f` for floating-point values or `s` for strings. See Table 3 for the most important formats

printf Format Flags

Table 2 Format Flags

Flag	Meaning	Example
-	Left alignment	1.23 followed by spaces
0	Show leading zeroes	001.23
+	Show a plus sign for positive numbers	+1.23
(	Enclose negative numbers in parentheses	(1.23)
,	Show decimal separators	12,300
^	Convert letters to uppercase	1.23E+1

printf Format Types

Table 3 Format Types

Code	Type	Example
d	Decimal integer	123
f	Fixed floating-point	12.30
e	Exponential floating-point	1.23e+1
g	General floating-point (exponential notation is used for very large or very small values)	12.3
s	String	Tax:

Special Topic

- Reading an Entire File

- You can read an entire file into a list of lines, or into a single string
 - Use the `Files` and `Paths` classes

```
String filename = . . . ;  
List<String> lines = Files.readAllLines(Paths.get(filename));  
String content =  
    new String(Files.readAllBytes(Paths.get(filename)));
```

Command Line Arguments

- Text based programs can be 'parameterized' by using command line arguments
 - Filename and options are often typed after the program name at a command prompt:

```
>java ProgramClass -v input.dat
```



```
public static void main(String[] args)
```

- Java provides access to them as an array of `Strings` parameter to the main method named `args`

```
args[0]: "-v"  
args[1]: "input.dat"
```

- The `args.length` variable holds the number of `args`
- Options (switches) traditionally begin with a dash '-'

Steps to Processing Text Files

- Read two country data files, `worldpop.txt` and `worldarea.txt`.
- Write a file `world_pop_density.txt` that contains country names and population densities with the country names aligned left and the numbers aligned right.

Afghanistan	50.56
Akrotiri	127.64
Albania	125.91
Algeria	14.18
American Samoa	288.92
...	

Steps to Processing Text Files

- 1) Understand the Processing Task
 - Process 'on the go' or store data and then process?
- 2) Determine input and output files
- 3) Choose how you will get file names
- 4) Choose line, word or character-based input processing
 - If all data is on one line, normally use line input
- 5) With line-oriented input, extract required data
 - Examine the line and plan for whitespace, delimiters...
- 6) Use methods to factor out common tasks

Processing Text Files: Pseudocode

- Step 1: Understand the Task

While there are more lines to be read

 Read a line from each file

 Extract the country name

 population = number following the country name in
 the line from the first file

 area = number following the country name in the line
 from the second file

 If area != 0

 density = population / area

 Print country name and density

Afghanistan	50.56
Akrotiri	127.64
Albania	125.91
Algeria	14.18
American Samoa	288.92
...	

Exception Handling

- There are two aspects to dealing with run-time program errors:
 - Detecting Errors
 - This is the easy part. You can 'throw' an exception

Use the throw statement to
signal an exception

```
if (amount > balance)
{
    // Now what?
}
```

- Handling Errors
 - This is more complex. You need to 'catch' each possible exception and react to it appropriately
- Handling recoverable errors can be done:
 - Simply: exit the program
 - User-friendly: Ask the user to correct the error

Syntax 7.1 Throwing an Exception

- When you throw an exception, you are throwing an object of an exception class
 - Choose wisely!
 - You can also pass a descriptive `String` to most exception objects

Syntax `throw exceptionObject;`

A new exception object is constructed, then thrown.

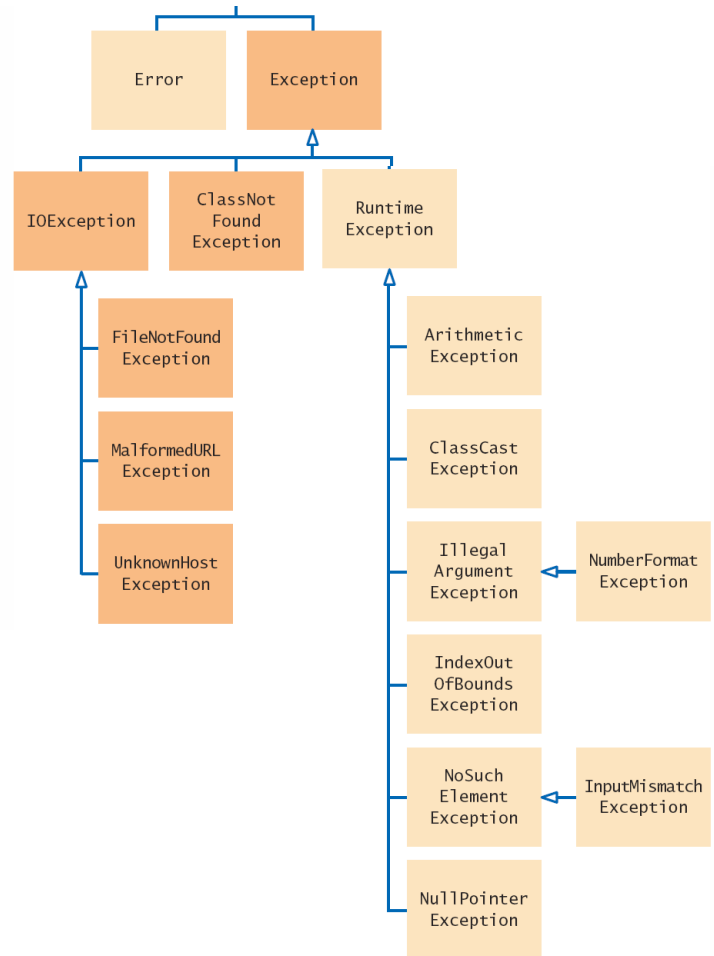
```
if (amount > balance)
{
    throw new IllegalArgumentException("Amount exceeds balance");
}
balance = balance - amount;
```

Most exception objects can be constructed with an error message.

This line is not executed when the exception is thrown.

Exception Classes

- Partial hierarchy of exception classes
 - More general are above
 - More specific are below
 - Darker are Checked exceptions



Catching Exceptions

- Exceptions that are thrown must be 'caught' somewhere in your program

```
try
{
    String filename = . . .;
    Scanner in = new Scanner(new File(filename));
    String input = in.next();
    int value = Integer.parseInt(input);
    . . .
}
catch (IOException exception)
{
    exception.printStackTrace();
}
catch (NumberFormatException exception)
{
    System.out.println("Input was not a number");
}
```

Surround method calls that can throw exceptions with a 'try block'.

FileNotFoundException

NoSuchElementException

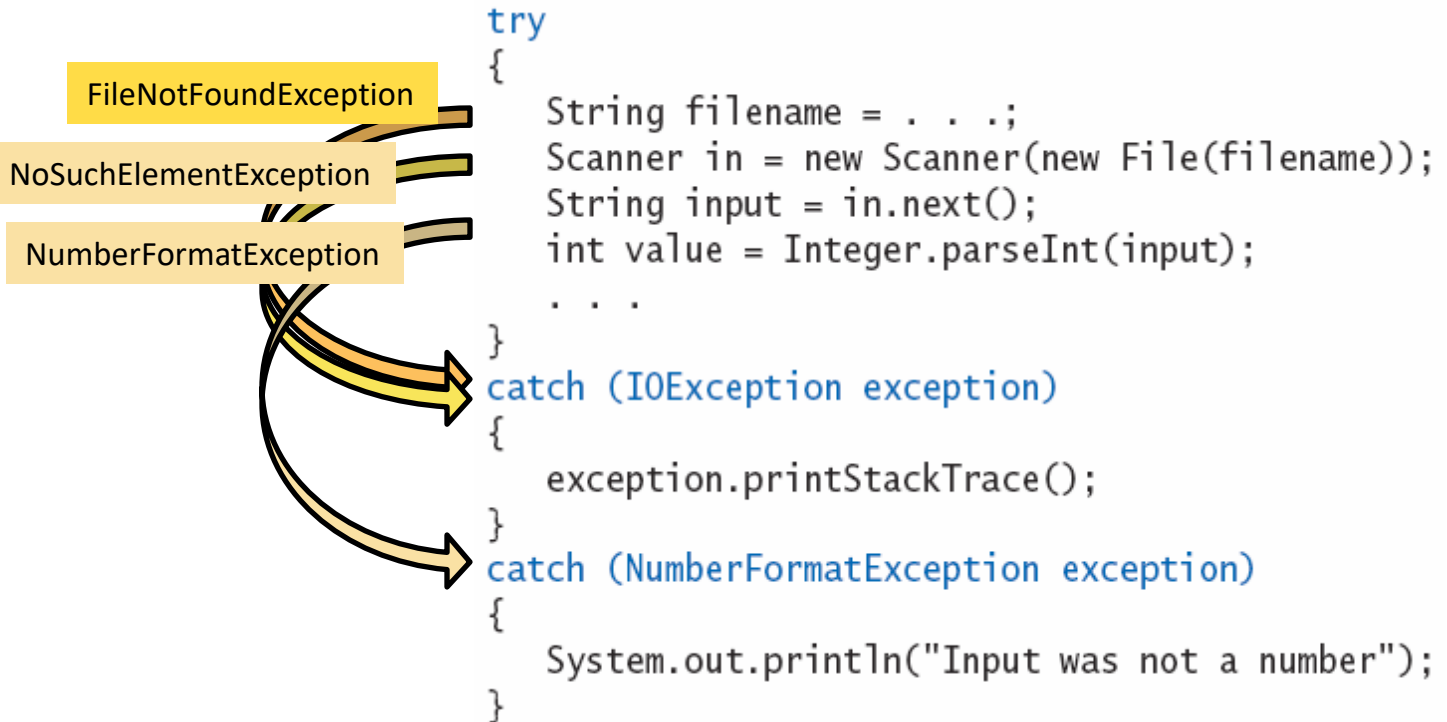
NumberFormatException

Write 'catch blocks' for each possible exception.

It is customary to name the exception parameter either 'e' or 'exception' in the catch block.

Catching Exceptions

- When an exception is detected, execution 'jumps' immediately to the first matching `catch` block
 - `IOException` catches `FileNotFoundException` and `NoSuchElementException` is not caught



Syntax 7.2 Catching Exceptions

Syntax

```
try
{
    statement
    statement
    . . .
}
catch (ExceptionClass exceptionObject)
{
    statement
    statement
    . . .
}
```

This constructor can throw a `FileNotFoundException`.

When an `IOException` is thrown, execution resumes here.

Additional catch clauses can appear here. Place more specific exceptions before more general ones.

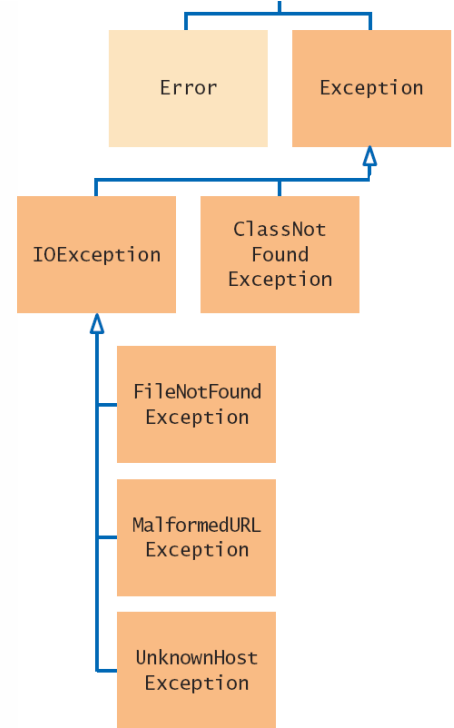
```
try
{
    Scanner in = new Scanner(new File("input.txt"));
    String input = in.next();
    process(input);
}
catch (IOException exception)
{
    System.out.println("Could not open input file");
}
catch (Exception except)
{
    System.out.println(except.getMessage());
}
```

This is the exception that was thrown.

A `FileNotFoundException` is a special case of an `IOException`.

Checked Exceptions

- Throw/catch applies to three types of exceptions:
 - **Error:** Internal Errors
 - not considered here
 - **Unchecked:** RunTime Exceptions
 - Caused by the programmer
 - Compiler **does not check** how you handle them
 - **Checked:** All other exceptions
 - Not the programmer's fault
 - Compiler **checks** to make sure you handle these
 - Shown darker in Exception Classes



Syntax 7.3 The **throws** Clause

- Methods that use other methods that may throw exceptions must be declared as such
 - Declare all **checked** exceptions a method throws
 - You may also list **unchecked** exceptions

Syntax *modifiers returnType methodName(parameterType parameterName, . . .)*
 throws ExceptionClass, ExceptionClass, . . .

```
public static String readData(String filename)
    throws FileNotFoundException, NumberFormatException
```

You **must** specify all checked exceptions
that this method may throw.

You may also list unchecked exceptions.

- Coded on a method's header
- Signals a caller of your method that it may encounter the named exceptions
- Provides a way to pass an exception up the calling chain to a more appropriate handler
- Ensures exceptions are not lost

The **throws** Clause

- If a method handles a checked exception internally, it will no longer throw the exception.
 - The method does not need to declare it in the throws clause
- Declaring exceptions in the **throws** clause 'passes the buck' to the calling method to handle it or pass it along.

Closing Resources

- Special treatment must be given to resources that must be closed
- Consider the code below:

```
PrintWriter out = new PrintWriter(filename);  
writeData(out);  
out.close(); // May never get here
```

- The `try-with-resources` statement calls the `close` method of the named resource automatically when the try block is completed
- Named resources must implement the `AutoCloseable` interface

Syntax 7.3 try-with-resources

Syntax `try` (*Type1 variable1 = expression1; Type2 variable2 = expression2; . . .*)
 {
 . . .
 }

This code may
throw exceptions.

```
try (PrintWriter out = new PrintWriter(filename))  
{  
    writeData(out);  
}
```

Implements the
AutoCloseable
interface.

At this point, `out.close()` is called,
even when an exception occurs.

Programming Tip

- Throw Early
 - When a method detects a problem that it cannot solve, it is better to throw an exception rather than try to come up with an imperfect fix
- Catch Late
 - Conversely, a method should only catch an exception if it can really remedy the situation
 - Otherwise, the best remedy is simply to have the exception propagate to its caller, allowing it to be caught by a competent handler

Programming Tip

- Do Not Squelch Exceptions

- When you call a method that throws a checked exception and you haven't specified a handler, the compiler complains

- It is tempting to write a 'do-nothing' catch block to 'squelch' the compiler and come back to the code later. **Bad Idea!**

- Exceptions were designed to transmit problem reports to a competent handler

- Installing an incompetent handler simply hides an error condition that could be serious

Programming Tip

- Do Throw Specific Exceptions
 - When throwing an exception, choose an exception class that describes the situation as closely as possible
 - You can provide a new exception class yourself for particular errors

Special Topic

▪ Assertions

- An assertion is a condition that you believe to be true at all times in a particular program location
- An assertion check tests whether an assertion is true
- When an assertion is correct, the program works in the normal way
- When an assertion fails, and assertion checking is enabled, an `AssertionError` is thrown, causing the program to terminate

```
public double deposit(double amount)
{
    assert amount >= 0;
    balance = balance + amount;
}
```

By default, assertion checking is disabled when you execute a program. To execute a program with assertion checking turned on, use this command:

```
java -enableassertions
MainClass
```


Special Topic

- `finally` is an optional clause in a `try/catch` block
 - Used to take some action in a method whether an exception is thrown or not
 - Rarely required because most Java library classes that require cleanup implement the `AutoCloseable` interface

```
public void printOutput(String filename) throws IOException
{
    PrintWriter out = new PrintWriter(filename);
    try
    {
        writeData(out);    // Method may throw an I/O Exception
    }
    finally
    {
        out.close();
    }
}
```

Once a try block is entered, the statements in a `finally` clause are guaranteed to be executed, whether or not an exception is thrown.

Handling Input Errors

- File Reading Application Example

- Goal: Read a file of data values

- First line is the count of values

- Remaining lines have values

- Risks:

- The file may not exist

- `Scanner` constructor will throw an exception

- `FileNotFoundException`

- The file may have data in the wrong format

- Doesn't start with a count

- `NoSuchElementException`

- Too many items (count is too low)

- `IOException`

```
3
1.45
-2.1
0.05
```

Handling Input Errors: main

▪Outline for method with all exception handling

```
boolean done = false;
while (!done)
{
    try
    {
        // Prompt user for file name
        double[] data = readFile(filename);    // May throw exceptions
        // Process data
        done = true;
    }
    catch (FileNotFoundException exception)
    {
        System.out.println("File not found.");    }
    catch (NoSuchElementException exception)
    {
        System.out.println("File contents invalid.");    }
    catch (IOException exception)
    {
        exception.printStackTrace();    }
}
```

Handling Input Errors: readFile

- Constructs the `Scanner` object and calls the `readData` method
- Does not handle exceptions
- Errors with the file are passed to the caller
- The file is closed even when an exception occurs
- Does not throw `FileNotFoundException` because that is a special case of `IOException`

```
public static double[] readFile(String filename) throws IOException
{
    File inFile = new File(filename);
    try (Scanner in = new Scanner(inFile))
    {
        return readData(in);
    }
}
```

Handling Input Errors: readData

- No exception handling (no `try` or `catch` clauses)
- `throw` creates an `IOException` object and exits
- unchecked `NoSuchElementException` can occur

```
public static double[] readData(Scanner in) throws IOException
{
    int numberOfValues = in.nextInt();    // NoSuchElementException
    double[] data = new double[numberOfValues];
    for (int i = 0; i < numberOfValues; i++)
    {
        data[i] = in.nextDouble();        // NoSuchElementException
    }
    if (in.hasNext())
    {
        throw new IOException("End of file expected");
    }
    return data;
}
```